

PPS Developer Guide

Version v0.30.0

Table of Contents

1. Start developing for PPS	2
2. Creating a custom TMSc reconstructor	4
3. Extending Data Analysis	7
4. PPS meta-models	10
4.1. Contents of architecture	10
4.1.1. Types	10
4.1.1.1. Interface ArchitectureModel	10
4.1.1.2. Abstract Class NamedArchitectureElement	10
4.2. Contents of deployed	11
4.2.1. Types	12
4.2.1.1. Class Host	12
4.3. Contents of implemented	12
4.3.1. Enumerations	13
4.3.1.1. FunctionParameterKind	13
4.3.2. Types	13
4.3.2.1. Class Function	13
4.3.2.2. Class FunctionParameter	14
4.3.2.3. Class IPCClientFunction	14
4.3.2.4. Abstract Class IPCFunction	15
4.3.2.5. Class IPCServerFunction	15
4.4. Contents of instantiated	15
4.4.1. Types	16
4.4.1.1. Class Executor	16
4.4.1.2. Interface ExecutorGroup	17
4.5. Contents of metric	17
4.5.1. Types	18
4.5.1.1. Class Metric	18
4.5.1.2. Class MetricCategory	19
4.5.1.3. Class MetricInstance	19
4.5.1.4. Class MetricModel	20
4.6. Contents of properties	20
4.6.1. Data Types	21
4.6.1.1. EPropertyValue	21
4.6.2. Types	21
4.6.2.1. Abstract Class PropertiesContainer	21
4.6.2.2. Class PropertyMapEntry	22
4.7. Contents of specified	23
4.7.1. Types	24

4.7.1.1. Class Component	24
4.7.1.2. Class Interface	25
4.7.1.3. Class Operation	25
4.8. Contents of tmsc	26
4.8.1. Data Types	28
4.8.1.1. EDuration	28
4.8.1.2. EID	29
4.8.1.3. ETimestamp	29
4.8.2. Types	30
4.8.2.1. Abstract Class Dependency	30
4.8.2.2. Class DomainDependency	33
4.8.2.3. Class EntryEvent	33
4.8.2.4. Abstract Class Event	34
4.8.2.5. Class Execution	36
4.8.2.6. Class ExitEvent	39
4.8.2.7. Class FullScopeTMSC	39
4.8.2.8. Class FunctionArgumentMapEntry	41
4.8.2.9. Interface ITMSC	42
4.8.2.10. Interface ITimeRange	42
4.8.2.11. Abstract Class Interval	44
4.8.2.12. Class Lifeline	45
4.8.2.13. Class LifelineSegment	47
4.8.2.14. Class Measurement	48
4.8.2.15. Class Message	48
4.8.2.16. Class MessageControl	49
4.8.2.17. Class Reply	49
4.8.2.18. Class Request	50
4.8.2.19. Class ScopedTMSC	50
4.8.2.20. Abstract Class TMSC	51

This developer guide describes how to tailor and extend the Platform Performance Suite (PPS) for a specific domain.



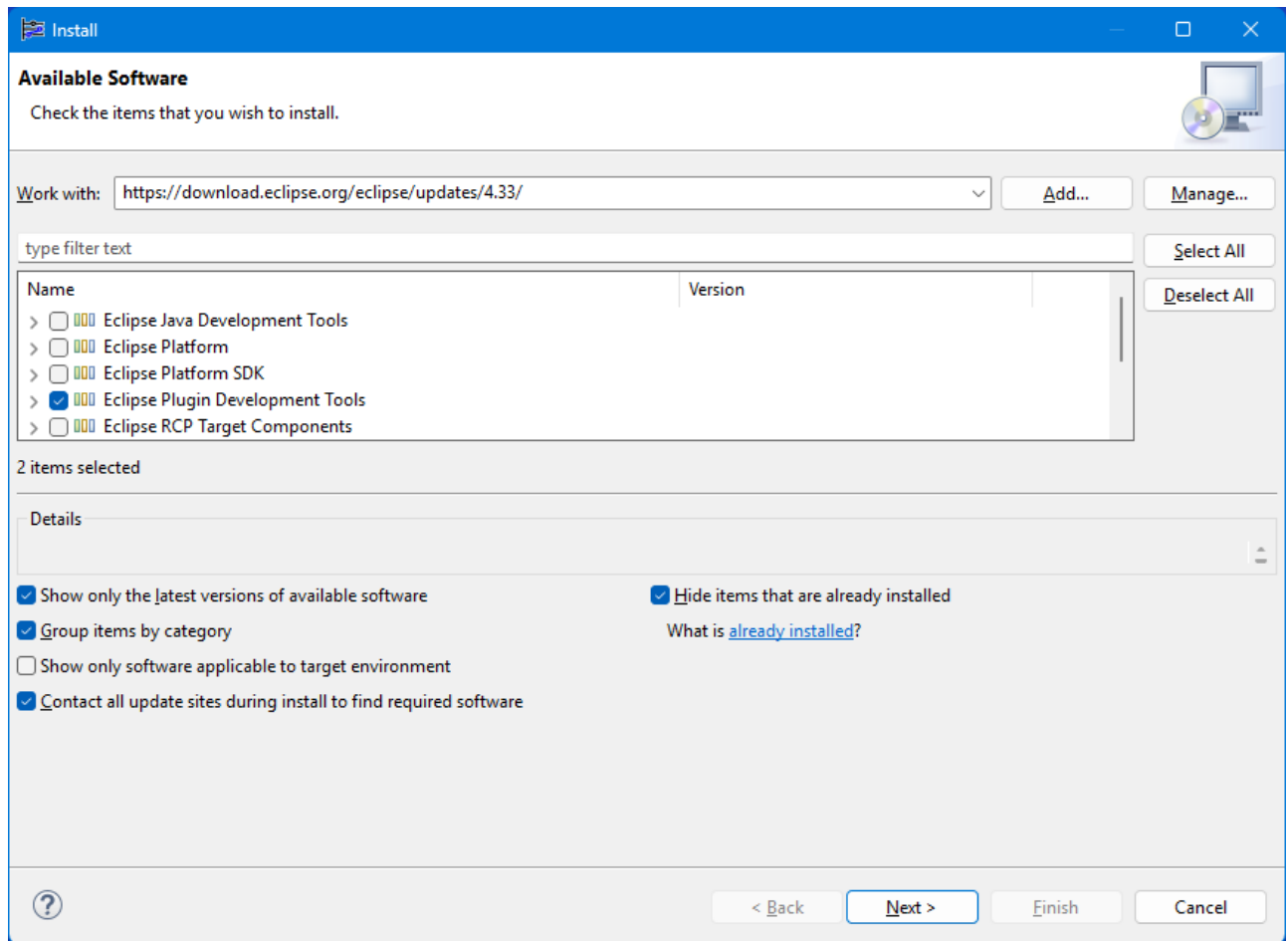
Reading this developer guide requires intermediate to advanced knowledge about programming for the Eclipse Modeling Framework (EMF). A good book to read as reference is [EMF: Eclipse Modeling Framework](#).



Chapter 1. Start developing for PPS

So, you got a binary distribution of PPS and you want to tailor it for your own domain specific use? This section describes how to convert your PPS to an Eclipse PDE to customize PPS.

1. First extract the PPS distribution, choose a PPS version (i.e., windows or linux), extract it and start it, e.g. `pps.exe`.
2. Now use the **Help > Install New Software...** menu to install the **Eclipse Plugin Development Tools** from the Eclipse 2024-09 (i.e., 4.33) update site: <https://download.eclipse.org/eclipse/updates/4.33/>



3. Continue the wizard using the **[Next]** button and finally the **[Finish]** button. When asked, choose to restart the Eclipse IDE.
4. After restart install the PPS sources by using the **Help > Install New Software...** menu. Select the **PPS** category from the PPS (e.g., v0.30.0) update site: <https://tno.github.io/PPS/update-site/v0.30.0/>



Please make sure to select the right update site version for your installation.
You can see your installed PPS version from the **Help > About PPS vX.X** menu.

5. Continue the wizard using the **[Next]** button and finally the **[Finish]** button. When asked, choose to restart the Eclipse IDE.

Your environment is now ready for extending PPS, e.g., [adding a custom TMSC reconstructor](#) or [extending its data analysis](#).



To start a PPS including your extensions, simply select one of your projects in the project/package explorer view and use its context menu (i.e., right mouse click) to **Run As › Eclipse Application**. A new Eclipse instance (i.e. an Eclipse runtime instance) will be executed containing all PPS features including your own features.

Chapter 2. Creating a custom TMSC reconstructor

The TMSC textual syntax allows you to create small example traces, to explore the PPS features. The textual syntax is not developed for large traces as it will suffer from scalability issues. Besides that, it is assumed that when using PPS in another domain, traces in another format are already available. In both cases it is advised to write your own *reconstructor* that converts these trace directly into a [TMSC meta-model](#) instance. This section gives an example on how to create such a custom reconstructor.

The custom TMSC reconstructor example is included in the PPS distribution. The example can be easily imported into your development environment by:

1. Use the **File › Import...** menu and then choose to import **Plug-in Development › Plug-ins and Fragments** from the tree.
2. On the next wizard page, select the **Projects with source folders** in the **Import As** section.
3. On the next wizard page filter the available plug-ins and fragments by **nl.esi.pps.tmsc.reconstruct** and click the **[Add All →]** button.
 - Now **[Finish]** the wizard to import the projects.



For more information on creating custom EMF resources or using extension points, please read the Eclipse and EMF documentation.



You can test the example by creating a ***.tmsctrace** file in a project within the Eclipse runtime instance and then use its **Reconstruct TMSC** context menu to create a TMSC for the trace.

First choose a syntax for your trace (e.g., **tmsctrace** in our example) and register it by means of the **org.eclipse.emf.ecore.extension_parser** extension point in the **plugin.xml** of your plugin.

Listing 1. plugin.xml of nl.esi.pps.tmsc.reconstruct

```
<?xml version="1.0" encoding="UTF-8"?>
<!--

    Copyright (c) 2018-2025 TNO and Contributors to the GitHub community

    This program and the accompanying materials are made available
    under the terms of the MIT License which is available at
    https://opensource.org/licenses/MIT

    SPDX-License-Identifier: MIT

-->
<?eclipse version="3.4"?>
<plugin>
```

```

<extension
    point="org.eclipse.emf.ecore.extension_parser">
    <parser
        class="nl.esi.pps.tmsc.reconstruct.TmscTraceResourceFactory"
        type="tmsctrace">
    </parser>
</extension>
</plugin>

```

The registered `nl.esi.pps.tmsc.reconstruct.TmscTraceResourceFactory` extends `org.eclipse.emf.ecore.resource.Resource.Factory` and creates an `nl.esi.pps.tmsc.reconstruct.TmscTraceResource` upon request. In our example, this resource defines an interface `nl.esi.pps.tmsc.reconstruct.TmscTraceEvent` between the parser and the reconstructor to separate the concerns of parsing and model forming. The call-outs after the example provide some more information.

Listing 2. `nl.esi.pps.tmsc.reconstruct.TmscTraceResource`

```

/*
 * Copyright (c) 2018-2025 TNO and Contributors to the GitHub community
 *
 * This program and the accompanying materials are made available
 * under the terms of the MIT License which is available at
 * https://opensource.org/licenses/MIT
 *
 * SPDX-License-Identifier: MIT
 */
package nl.esi.pps.tmsc.reconstruct;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.LineNumberReader;
import java.util.Map;

import org.eclipse.emf.common.util.URI;
import org.eclipse.emf.ecore.resource.impl.ResourceImpl;

public class TmscTraceResource extends ResourceImpl { ①
    public TmscTraceResource() {
        super();
    }

    public TmscTraceResource(URI uri) {
        super(uri);
    }

    @Override
    protected void doLoad(InputStream inputStream, Map<?, ?> options) throws
    IOException {

```

```

TmscTraceReconstructor reconstructor = new TmscTraceReconstructor(); ②
reconstructor.preReconstruct();

try (LineNumberReader reader = new LineNumberReader(new InputStreamReader
(inputStream))) {
    try {
        String line;
        while ((line = reader.readLine()) != null) {
            TmscTraceEvent traceEvent = TmscTraceParser.parseLine(line); ③
            reconstructor.reconstruct(traceEvent);
        }
    } catch (RuntimeException e) {
        throw new IOException("Failed to parse trace at line " + reader
.getLineNumber(), e);
    }
}

reconstructor.postReconstruct(); ④
getContents().add(reconstructor.getTmsc());
getContents().add(reconstructor.getArchitecture());
}
}

```

- ① Extending `org.eclipse.emf.ecore.resource.impl.ResourceImpl` allows you to implement the `doLoad(InputStream, Map)` method. If you also need to be able to save a TMSC model instance to your trace format, please implement the `doSave(OutputStream, Map)` method.
- ② At the beginning of the load, the `TmscReconstructor` is initialized by calling its `preReconstruct()` method. This will prepare the `Contents of architecture` and `Contents of tmsc`.
- ③ For every line in the trace, a `TmscTraceEvent` is created by the `TmscTraceParser` and this event is fed to the `TmscReconstructor` by calling its `reconstruct(TmscTraceEvent)` method.
- ④ The TMSC model instance is *well-formed* by calling the `postReconstruct()` method on the `TmscReconstructor`. As part of the `postReconstruct`, some generic `nl.esi.pps.tmsc.util.TmscRefinements` are called, please see the `TmscReconstructor` implementation. Finally, the reconstructed TMSC and architecture are added to the resource contents.

Chapter 3. Extending Data Analysis

The [Data Analysis view for the TMSC graphical viewer](#) shows either a time series or a histogram of the 'duration' for all 'siblings' of the selected element. The 'duration' of an element can be any time-related aspect of an element, e.g. its absolute (observed) duration or its time-bound. The 'siblings' of an element are all occurrences in the model for which their 'duration' is comparable, in other words, it makes sense to compare them. This already sounds very abstract and besides that, it is not known which elements are added to the TMSC model by means of extensions (e.g. analysis or customer specific model extensions). For this reason an extensible approach is designed to provide the input for data analysis.

This design is implemented by means of reusing the EMF Edit ItemProvider adapter factories extension point in combination with our own `nl.esi.pps.tmsc.edit.dataAnalysis` extension point. EMF adapters are a very powerful technique and EMF Edit's ItemProviders are designed to automatically add UI adapters to an EMF model instance. For more information on adapters and adapter factories, see <https://eclipsesource.com/blogs/2013/01/29/emf-itemprovider-magic/> or the book [EMF: Eclipse Modeling Framework](#).

The `nl.esi.pps.tmsc.edit.dataAnalysis` extension point adds a flexible approach on top, that allows to register an `nl.esi.pps.tmsc.provider.dataanalysis.IDataAnalysisItemContentProvider` per `EClass`. This item provider needs to be registered to the EMF framework in order to be found by the data analysis view. For this, an `nl.esi.pps.tmsc.provider.dataanalysis.internal.DataAnalysisItemContentProvider` is registered as EMF Edit ItemProvider via the `nl.esi.pps.tmsc.provider.dataanalysis.DataAnalysisItemContentProviderAdapterFactory`, see the `plugin.xml` of `nl.esi.pps.tmsc.edit`. We only register the `nl.esi.pps.tmsc.provider.dataanalysis.IDataAnalysisItemContentProvider` as `supportedTypes` and implement the default `create` method to returning an instance of the `nl.esi.pps.tmsc.provider.dataanalysis.internal.DataAnalysisItemContentProvider`. We can use a single instance as our implementation is stateless. This instance is disposed when the adapter factory is disposed.



Registering an EMF Edit ItemProvider only needs to be done once per language that wants to support data analysis!

The `nl.esi.pps.tmsc.provider.dataanalysis.IDataAnalysisItemContentProvider` interface defines the API to populate the data analysis views. The API is explained in the JavaDoc of the interface. All methods take an `Object` as argument, this is the model element for which the data is requested. First the `Set<String> getConfigurations(Object)` will be invoked. This method should return all supported data analysis configurations for the model element. An example of supporting different configurations is implemented in `nl.esi.pps.tmsc.provider.ext.ExecutionExtItemProvider` where 'siblings' can either be executions on the same `Lifeline` or executions executed by the same `Component`. The configurations will appear as tabs in the data analysis view. In all other methods, both the model element as the user selected configuration at least will be provided.



Within Eclipse, use `Ctrl` + `Shift` + `T` to open a type based on its name.

Listing 3. *nl.esi.pps.tmsc.provider.dataanalysis.IDataAnalysisItemContentProvider*

```
/**
 * This interface defines the required information to populate the Data Analysis
 * view of the TMSC editor.
 */
public interface IDataAnalysisItemContentProvider {
    /**
     * The default configuration for data analysis. If your data analysis only
     * supports a single configuration, its is recommended to use this
     * configuration.
     */
    static final String DEFAULT_CONFIGURATION = "Default";

    /**
     * Returns the supported data analysis configurations for {@code object}.
     */
    Set<String> getConfigurations(Object object);

    /**
     * Returns the title of the data analysis for {@code object} in the selected
     * {@code configuration}.
     */
    String getTitle(Object object, String configuration);

    /**
     * Returns the budget of the data analysis for {@code object} in the selected
     * {@code configuration}, or {@code null} if the budget is not applicable.
     */
    default Long getBudget(Object object, String configuration) {
        return null;
    }

    /**
     * Returns the siblings of {@code object} for the selected
     * {@code configuration}.
     */
    Iterable<?> getSiblings(Object object, String configuration);

    /**
     * Returns the duration of {@code sibling} in the selected
     * {@code configuration}. Please note that {@code sibling} can be any of the
     * elements as returned by {@code #getSiblings(Object, String)} or the
     * {@code object} itself.
     */
    Long getDuration(Object object, Object sibling, String configuration);

    /**
     * @return {@code true} if the bars in the time-series viewer should be colored
     *         based on a {@code #getCategory(Object, Object, String) category}.
     */
}
```

```

    default boolean isCategorized(Object object, String configuration) {
        return false;
    }

    /**
     * @return the category for the {@code sibling} in the time-series viewer.
     */
    default String getCategory(Object object, Object sibling, String configuration) {
        return DEFAULT_CONFIGURATION;
    }
}

```

The only thing left to do is to register the data analysis item content provider to the EMF framework, which is done by means of the `nl.esi.pps.tmsc.edit.dataAnalysis` extension point in the `plugin.xml`.

Listing 4. `plugin.xml` of `nl.esi.pps.tmsc.edit`

```

<extension point="org.eclipse.emf.edit.itemProviderAdapterFactory"> ①
    <factory
        uri="http://www.esi.nl/pps/tmsc"
        class=
        "nl.esi.pps.tmsc.provider.dataanalysis.DataAnalysisItemContentProviderAdapterFactory"
        supportedTypes=
        "nl.esi.pps.tmsc.provider.dataanalysis.IDataAnalysisItemContentProvider"/>
    </extension>
    <extension point="nl.esi.pps.tmsc.edit.dataAnalysis"> ②
        <DataAnalysis
            content_provider=
            "nl.esi.pps.tmsc.provider.dataanalysis.ExecutionDataAnalysisItemContentProvider"
            eclass_uri="http://www.esi.nl/pps/tmsc#Execution"
            id=
            "nl.esi.pps.tmsc.provider.dataanalysis.ExecutionDataAnalysisItemContentProvider">
        </DataAnalysis>
    </extension>

```

- ① An additional EMF-edit item provider adapter factory is registered for the <http://www.esi.nl/pps/tmsc> language to bind the `nl.esi.pps.tmsc.edit.dataAnalysis` extension point, see the `nl.esi.pps.tmsc.provider.dataanalysis.DataAnalysisItemContentProviderAdapterFactory` code for more information.
- ② The `nl.esi.pps.tmsc.provider.dataanalysis.ExecutionDataAnalysisItemContentProvider` is registered for the <http://www.esi.nl/pps/tmsc#Execution> EClass

Chapter 4. PPS meta-models

4.1. Contents of architecture

The architecture meta-model defines the minimal set of concepts that are required to define the **TMSC** meta-model. The architecture package contains sub-packages that represent the lifecycle stages of an architecture as inspired by Koziolok et al. ^[1], where the author presents a view on the software component lifecycle. This study considers 4 levels of lifecycle stages: the **specification**, **implementation**, deployment and runtime level. This architecture considers an additional distinction in the runtime level, which is divided in the (runtime) **instantiated** and (runtime) **stimulated** stage. The **TMSC** meta-model defines concepts that belong to the stimulated stage.

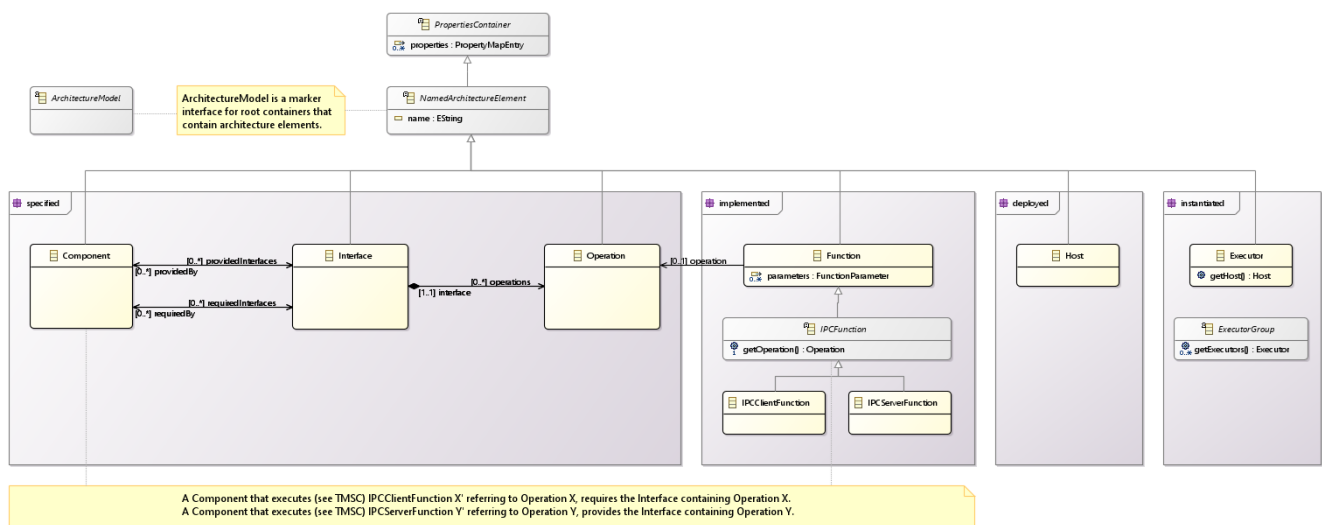


Figure 1. Diagram of the architecture meta-model

Ns Prefix

architecture

Ns URI

<http://www.esi.nl/pps/architecture>

4.1.1. Types

4.1.1.1. Interface ArchitectureModel

ArchitectureModel is a marker interface for root containers that contain architecture elements. Is is not required for an architecture to implement the interface, but it may speed up certain analyses.

Used at

- `tmsc.FullScopeTMSC.architectures`

4.1.1.2. Abstract Class NamedArchitectureElement

Super-types

- `properties.PropertiesContainer`

Sub-types

- `deployed.Host`
- `implemented.Function`
- `implemented.FunctionParameter`
- `implemented.IPCClientFunction`
- `implemented.IPCFunction`
- `implemented.IPCServerFunction`
- `instantiated.Executor`
- `specified.Component`
- `specified.Interface`
- `specified.Operation`

Attributes

Name	Type	Properties	Description
<code>name</code>	<code>EString</code>		

4.2. Contents of deployed

In the Deployed lifecycle stage, the implemented software components are built and bundled into binaries. At this stage, the binaries are placed into a filesystem from where they can be installed to the different hosts. The installation takes place during the transition from Deployed to Instantiated stage. During this transition the binaries are mapped (installed) to specific hosts. We consider for the sake of simplicity that the deployment of binaries onto hosts is static and performed before the system is initiated/started.

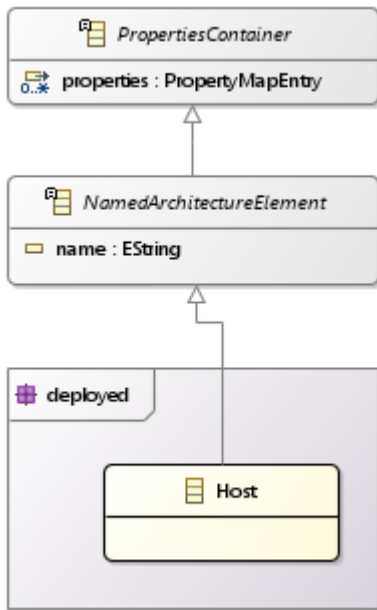


Figure 2. Diagram of the deployed lifecycle stage in the architecture meta-model

Ns Prefix

architecture_deployed

Ns URI

<http://www.esi.nl/pps/architecture/deployed>

4.2.1. Types

4.2.1.1. Class Host

Super-types

- [architecture.NamedArchitectureElement](#)
- [properties.PropertiesContainer](#)

Used at

- [instantiated.Executor.getHost\(\)](#)

4.3. Contents of implemented

In the Implementation lifecycle stage, the specified software components and interfaces are implemented. Depending on the software architecture, as well as the execution platform, a component may have more than one implementation, which may be related to a specific execution platform (i.e., programming language, type of processing unit, etc.) or other factors (i.e., quality of service, etc.)

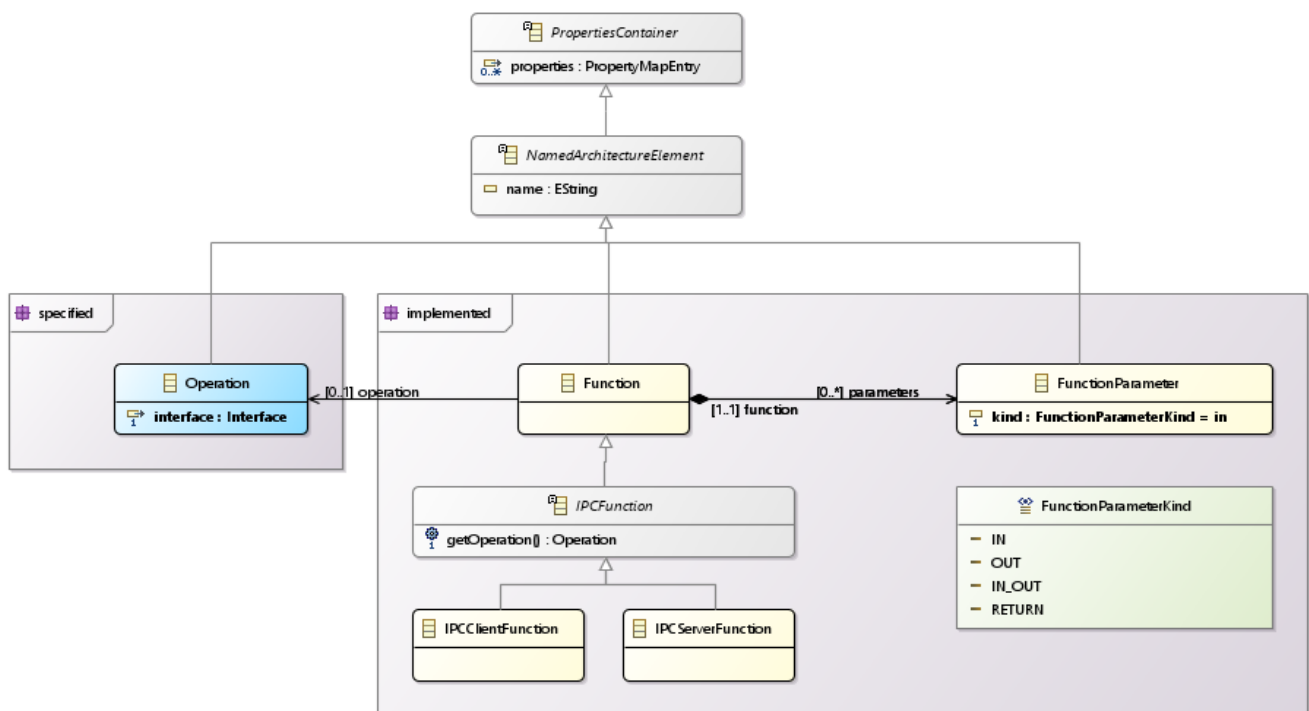


Figure 3. Diagram of the implemented lifecycle stage in the architecture meta-model

Ns Prefix

architecture_implemented

Ns URI

<http://www.esi.nl/pps/architecture/implemented>

4.3.1. Enumerations

4.3.1.1. FunctionParameterKind

Default Value

`in`

Literals

Name	Value	Literal	Description
IN	0	<code>in</code>	
OUT	1	<code>out</code>	
IN_OUT	2	<code>inout</code>	
RETURN	3	<code>return</code>	

Used at

- `implemented.FunctionParameter.kind`

4.3.2. Types

4.3.2.1. Class Function

Super-types

- `architecture.NamedArchitectureElement`
- `properties.PropertiesContainer`

Sub-types

- `implemented.IPCClientFunction`
- `implemented.IPCFunction`
- `implemented.IPCServerFunction`

Containments

Name	Type	Properties	Description
<code>parameters</code>	<code>implemented.FunctionParameter</code> <i>EOpposite:</i> <code>function</code>		

References

Name	Type	Properties	Description
operation implemented. IPCFunction. getOperation()	specified. Operation		

Used at

- implemented.FunctionParameter.function
- tmsc.EntryEvent.function
- tmsc.Event.function
- tmsc.Execution.function
- tmsc.ExitEvent.function

4.3.2.2. Class FunctionParameter

Super-types

- architecture.NamedArchitectureElement
- properties.PropertiesContainer

Attributes

Name	Type	Properties	Description
kind	implemented. FunctionParameterKind	[1]	

References

Name	Type	Properties	Description
function	implemented. Function EOpposite: parameters	non- resolveProxies container [1]	

Used at

- implemented.Function.parameters
- implemented.IPCClientFunction.parameters
- implemented.IPCFunction.parameters
- implemented.IPCServerFunction.parameters
- tmsc.FunctionArgumentMapEntry.key

4.3.2.3. Class IPCClientFunction

A Component that executes (see TMSC) IPCClientFunction X' referring to Operation X, requires the

Interface containing Operation X.

Super-types

- `architecture.NamedArchitectureElement`
- `implemented.Function`
- `implemented.IPCFunction`
- `properties.PropertiesContainer`

4.3.2.4. Abstract Class IPCFunction

Super-types

- `architecture.NamedArchitectureElement`
- `implemented.Function`
- `properties.PropertiesContainer`

Sub-types

- `implemented.IPCClientFunction`
- `implemented.IPCServerFunction`

Operations

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>getOperation()</code> <code>[] implemented.Function. operation</code>	<i>returns</i> <code>specified. Operation</code>	<code>[1]</code>	

4.3.2.5. Class IPCServerFunction

A Component that executes (see TMS) IPCServerFunction Y' referring to Operation Y, provides the Interface containing Operation Y.

Super-types

- `architecture.NamedArchitectureElement`
- `implemented.Function`
- `implemented.IPCFunction`
- `properties.PropertiesContainer`

4.4. Contents of instantiated

At instantiated stage the system is started, and the components are instantiated and start living in memory. Here, the runtime component takes form and is mapped on OS execution entities, i.e., on a specific thread which is mapped on specific process, etc. Communication ports are created and instantiated enabling communication among the runtime components. The OS execution entities,

i.e., processes and threads are mapped on specific processing unit, with specific scheduling policies, priorities, etc.

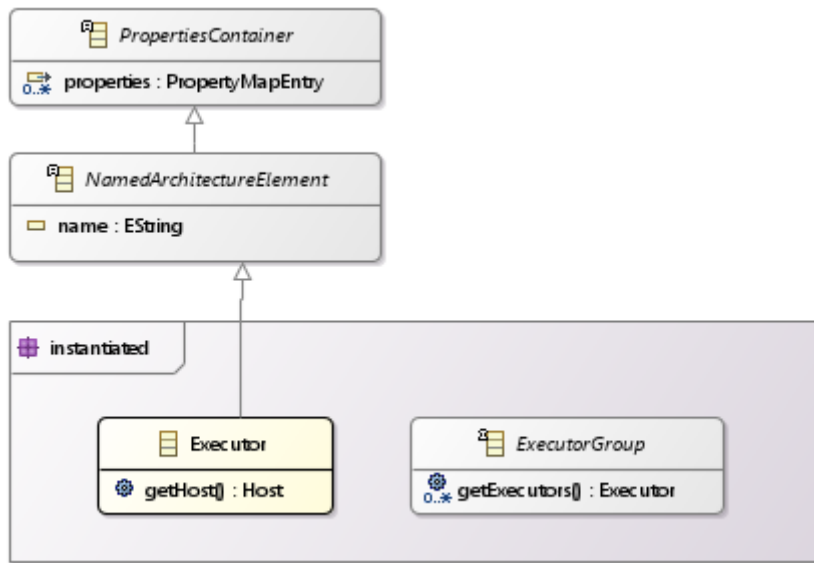


Figure 4. Diagram of the instantiated lifecycle stage in the architecture meta-model

Ns Prefix

architecture_instantiated

Ns URI

<http://www.esi.nl/pps/architecture/instantiated>

4.4.1. Types

4.4.1.1. Class Executor

Super-types

- [architecture.NamedArchitectureElement](#)
- [properties.PropertiesContainer](#)

Operations

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>getHost()</code>	returns <code>deployed.Host</code>		Typically an executor is deployed onto a host, but PPS doesn't make any assumptions on the kind of relation, hence this relation is merely implemented as an operation and it is up to the architecture implementer to define the relation. The default implementation of this method returns the container of type Host in the hierarchy of this Executor.

Used at

- `instantiated.ExecutorGroup.getExecutors()`
- `tmse.Lifeline.executor`

4.4.1.2. Interface ExecutorGroup

The ExecutorGroup allows to create logical groups of Executors that have a relation. When selecting an ExecutorGroup in the Outline view of the TMS viewer, all its Executors will be visualized.

Operations

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>getExecutors()</code>	returns <code>instantiated.Executor</code>	unique	Typically an ExecutorGroup contains Executors, but PPS doesn't make any assumptions on the kind of relation, hence this relation is merely implemented as an operation and it is up to the architecture implementer to define the relation. The default implementation of this method returns all first-level descendants of this ExecutorGroup that are of kind Executor, meaning if an Executor contains another Executor, the latter will not be included in the result.

4.5. Contents of metric

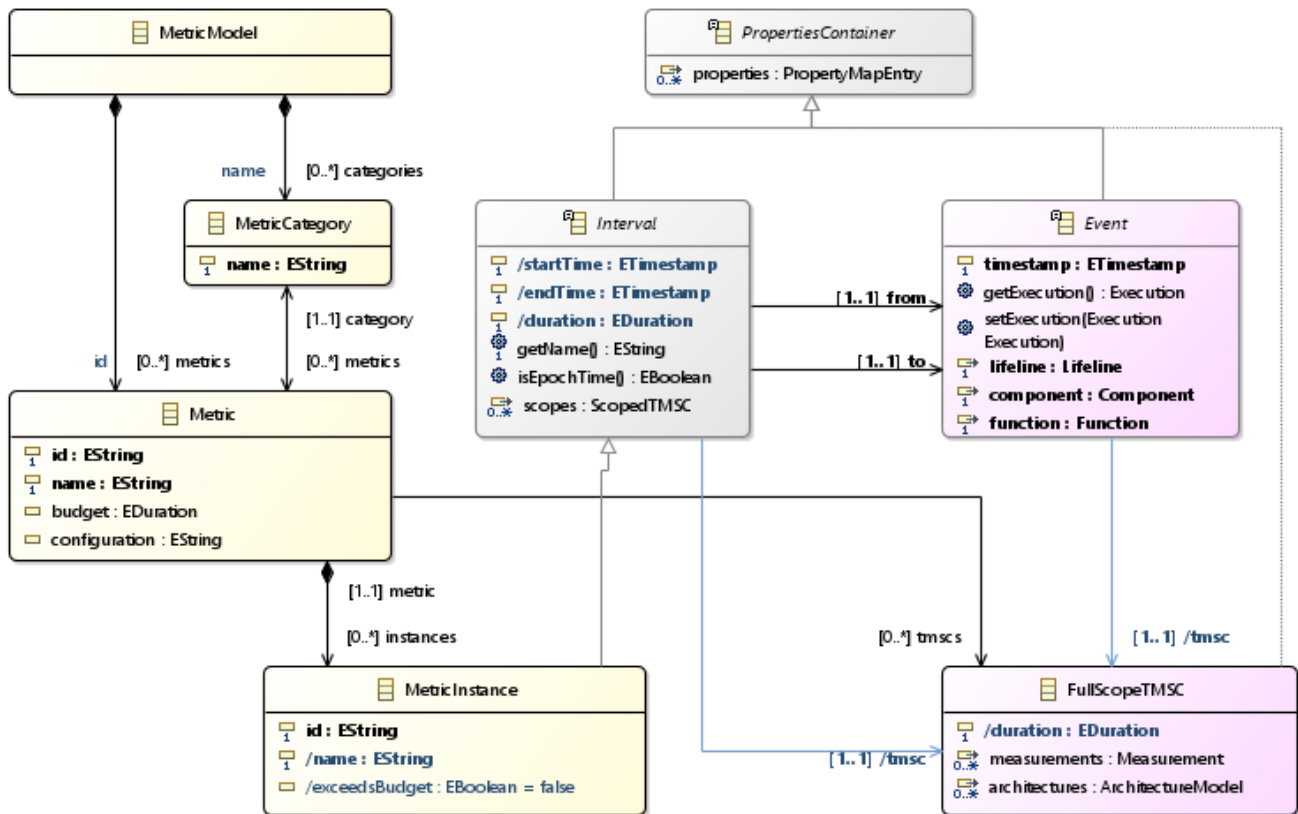


Figure 5. Diagram of the Metric meta-model

Ns Prefix

tmesc_metric

Ns URI

<http://www.esi.nl/tmesc/metric>

4.5.1. Types

4.5.1.1. Class Metric

Attributes

Name	Type	Properties	Description
budget	tmesc.EDuration		
configuration	EString		
id	EString	[1]	
name	EString	[1]	

Containments

Name	Type	Properties	Description
<code>instances</code>	<code>metric.MetricInstance</code> <i>EOpposite:</i> <code>metric</code>	<i>EKeys:</i> <code>id</code>	

References

Name	Type	Properties	Description
<code>category</code>	<code>metric.MetricCategory</code> <i>EOpposite:</i> <code>metrics</code>	<code>[1]</code>	
<code>tmcs</code>	<code>tmsc.FullScopeTMS</code>		The TMSs that were analyzed to resolve the Metric instances that are contained by this Metric.

Used at

- `metric.MetricCategory.metrics`
- `metric.MetricInstance.metric`
- `metric.MetricModel.metrics`

4.5.1.2. Class MetricCategory

Attributes

Name	Type	Properties	Description
<code>name</code>	<code>EString</code>	<code>[1]</code>	

References

Name	Type	Properties	Description
<code>metrics</code>	<code>metric.Metric</code> <i>EOpposite:</i> <code>category</code>		

Used at

- `metric.Metric.category`
- `metric.MetricModel.categories`

4.5.1.3. Class MetricInstance

Super-types

- `properties.PropertiesContainer`

- [tmsec.ITimeRange](#)
- [tmsec.Interval](#)

Attributes

Name	Type	Properties	Description
exceedsBudget	EBoolean	unchangeable derived transient volatile	
id	EString	[1]	
name tmsec.Interval.getName()	EString	[1] unchangeable derived transient volatile	

References

Name	Type	Properties	Description
metric	metric.Metric <i>EOpposite:</i> instances	container [1]	

Used at

- [metric.Metric.instances](#)

4.5.1.4. Class MetricModel

Containments

Name	Type	Properties	Description
categories	metric.MetricCategory	<i>EKeys:</i> name	
metrics	metric.Metric	<i>EKeys:</i> id	

4.6. Contents of properties

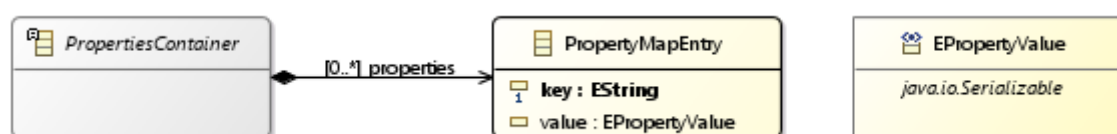


Figure 6. Diagram of the properties meta-model

Ns Prefix

properties

Ns URI

<http://www.esi.nl/properties>

4.6.1. Data Types

4.6.1.1. EPropertyValue

Instance Type Name

`java.io.Serializable`

Used at

- `properties.PropertyMapEntry.value`

4.6.2. Types

4.6.2.1. Abstract Class PropertiesContainer

The PropertiesContainer facilitates the seamless extension of specific modeling elements with additional attributes, that are required for specific type of analysis. Though this allows rapid prototyping or customer specific annotations, for every property that is added to the model the question should be raised if this property is a primary citizen of the meta-model. If so, the property should be promoted as attribute or reference in the meta-model.

Sub-types

- `architecture.NamedArchitectureElement`
- `deployed.Host`
- `implemented.Function`
- `implemented.FunctionParameter`
- `implemented.IPCClientFunction`
- `implemented.IPCFunction`
- `implemented.IPCServerFunction`
- `instantiated.Executor`
- `metric.MetricInstance`
- `specified.Component`
- `specified.Interface`
- `specified.Operation`
- `tmse.Dependency`
- `tmse.DomainDependency`
- `tmse.EntryEvent`
- `tmse.Event`
- `tmse.Execution`

- [tmsc.ExitEvent](#)
- [tmsc.FullScopeTMS](#)
- [tmsc.Interval](#)
- [tmsc.Lifeline](#)
- [tmsc.LifelineSegment](#)
- [tmsc.Measurement](#)
- [tmsc.Message](#)
- [tmsc.MessageControl](#)
- [tmsc.Reply](#)
- [tmsc.Request](#)
- [tmsc.ScopedTMS](#)
- [tmsc.TMS](#)

Containments

Name	Type	Properties	Description
properties	properties. PropertyMapEntry		

4.6.2.2. Class PropertyMapEntry

Instance Type Name

[java.util.Map\\$Entry](#)

Attributes

Name	Type	Properties	Description
key	EString	[1]	
value	properties. EPropertyValue		

Used at

- [architecture.NamedArchitectureElement.properties](#)
- [deployed.Host.properties](#)
- [implemented.Function.properties](#)
- [implemented.FunctionParameter.properties](#)
- [implemented.IPCClientFunction.properties](#)
- [implemented.IPCFunction.properties](#)
- [implemented.IPCServerFunction.properties](#)
- [instantiated.Executor.properties](#)
- [metric.MetricInstance.properties](#)

- `properties.PropertiesContainer.properties`
- `specified.Component.properties`
- `specified.Interface.properties`
- `specified.Operation.properties`
- `tmsc.Dependency.properties`
- `tmsc.DomainDependency.properties`
- `tmsc.EntryEvent.properties`
- `tmsc.Event.properties`
- `tmsc.Execution.properties`
- `tmsc.ExitEvent.properties`
- `tmsc.FullScopeTMSc.properties`
- `tmsc.Interval.properties`
- `tmsc.Lifeline.properties`
- `tmsc.LifelineSegment.properties`
- `tmsc.Measurement.properties`
- `tmsc.Message.properties`
- `tmsc.MessageControl.properties`
- `tmsc.Reply.properties`
- `tmsc.Request.properties`
- `tmsc.ScopedTMSc.properties`
- `tmsc.TMSc.properties`

4.7. Contents of specified

In this lifecycle stage, the specification of the software components takes place. Provided and required interfaces describe the contracts between the software components and encode the service specification. At this stage, the software interfaces are also specified by bundling operations that each of those implements.

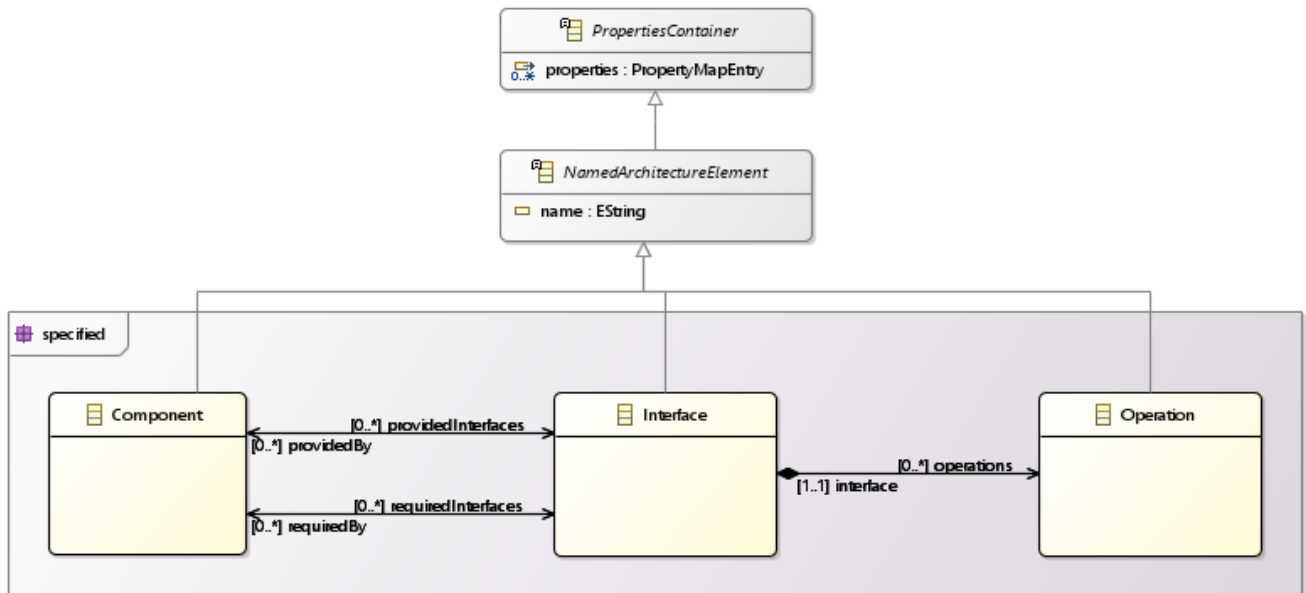


Figure 7. Diagram of the specified lifecycle stage in the architecture meta-model

Ns Prefix

architecture_specified

Ns URI

<http://www.esi.nl/pps/architecture/specified>

4.7.1. Types

4.7.1.1. Class Component

Super-types

- [architecture.NamedArchitectureElement](#)
- [properties.PropertiesContainer](#)

References

Name	Type	Properties	Description
providedInterfaces	specified.Interface <i>EOpposite:</i> providedBy		
requiredInterfaces	specified.Interface <i>EOpposite:</i> requiredBy		

Used at

- [specified.Interface.providedBy](#)
- [specified.Interface.requiredBy](#)

- [tmsec.EntryEvent.component](#)
- [tmsec.Event.component](#)
- [tmsec.Execution.component](#)
- [tmsec.ExitEvent.component](#)

4.7.1.2. Class Interface

Super-types

- [architecture.NamedArchitectureElement](#)
- [properties.PropertiesContainer](#)

Containments

Name	Type	Properties	Description
operations	specified.Operation <i>EOpposite:</i> interface		

References

Name	Type	Properties	Description
providedBy	specified.Component <i>EOpposite:</i> providedInterfaces		
requiredBy	specified.Component <i>EOpposite:</i> requiredInterfaces		

Used at

- [specified.Component.providedInterfaces](#)
- [specified.Component.requiredInterfaces](#)
- [specified.Operation.interface](#)

4.7.1.3. Class Operation

Super-types

- [architecture.NamedArchitectureElement](#)
- [properties.PropertiesContainer](#)

Name	Type	Properties	Description
interface	specified. Interface <i>EOpposite:</i> operations	container [1]	

Used at

- `implemented.Function.operation`
- `implemented.IPCClientFunction.operation`
- `implemented.IPCFunction.getOperation()`
- `implemented.IPCFunction.getOperation()`
- `implemented.IPCServerFunction.operation`
- `implemented.IPCFunction.getOperation()`
- `specified.Interface.operations`

4.8. Contents of tmsc

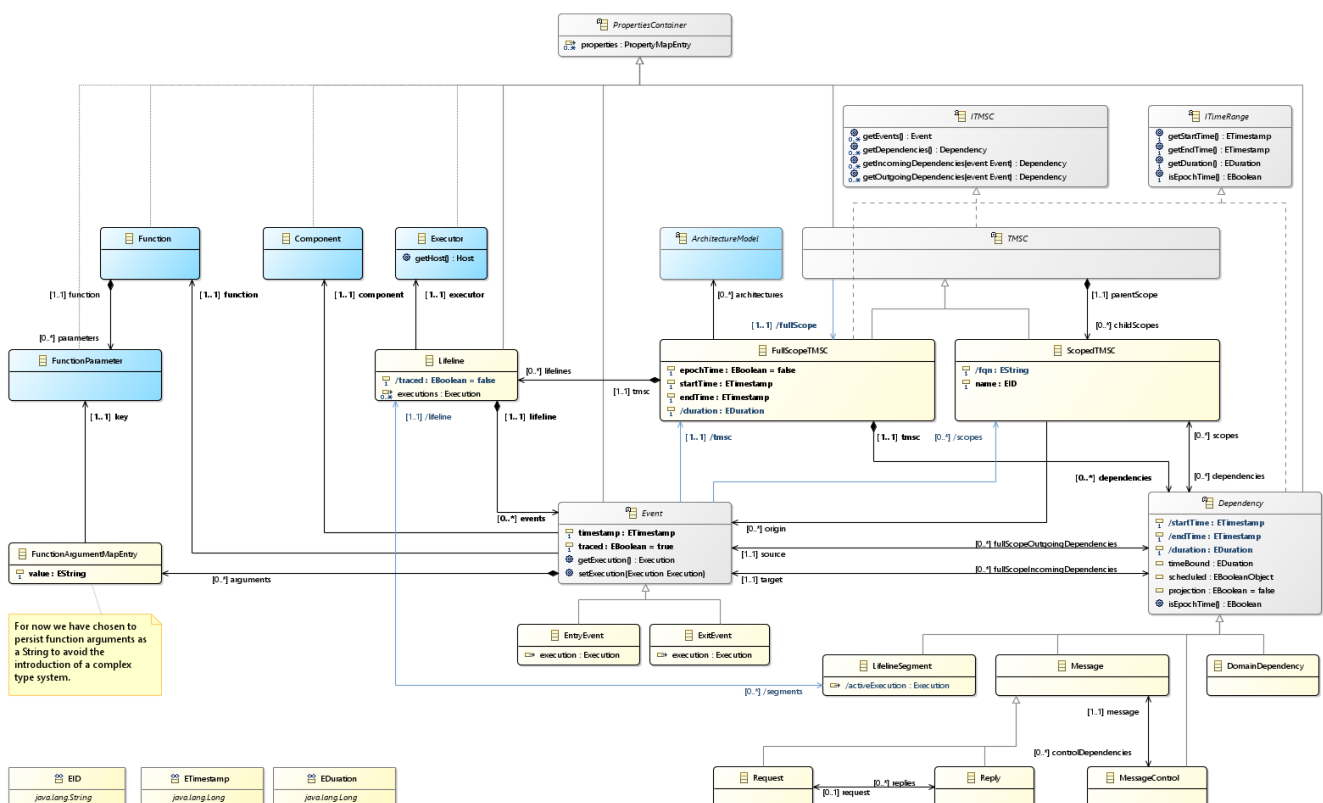


Figure 8. Diagram of the TMS meta-model

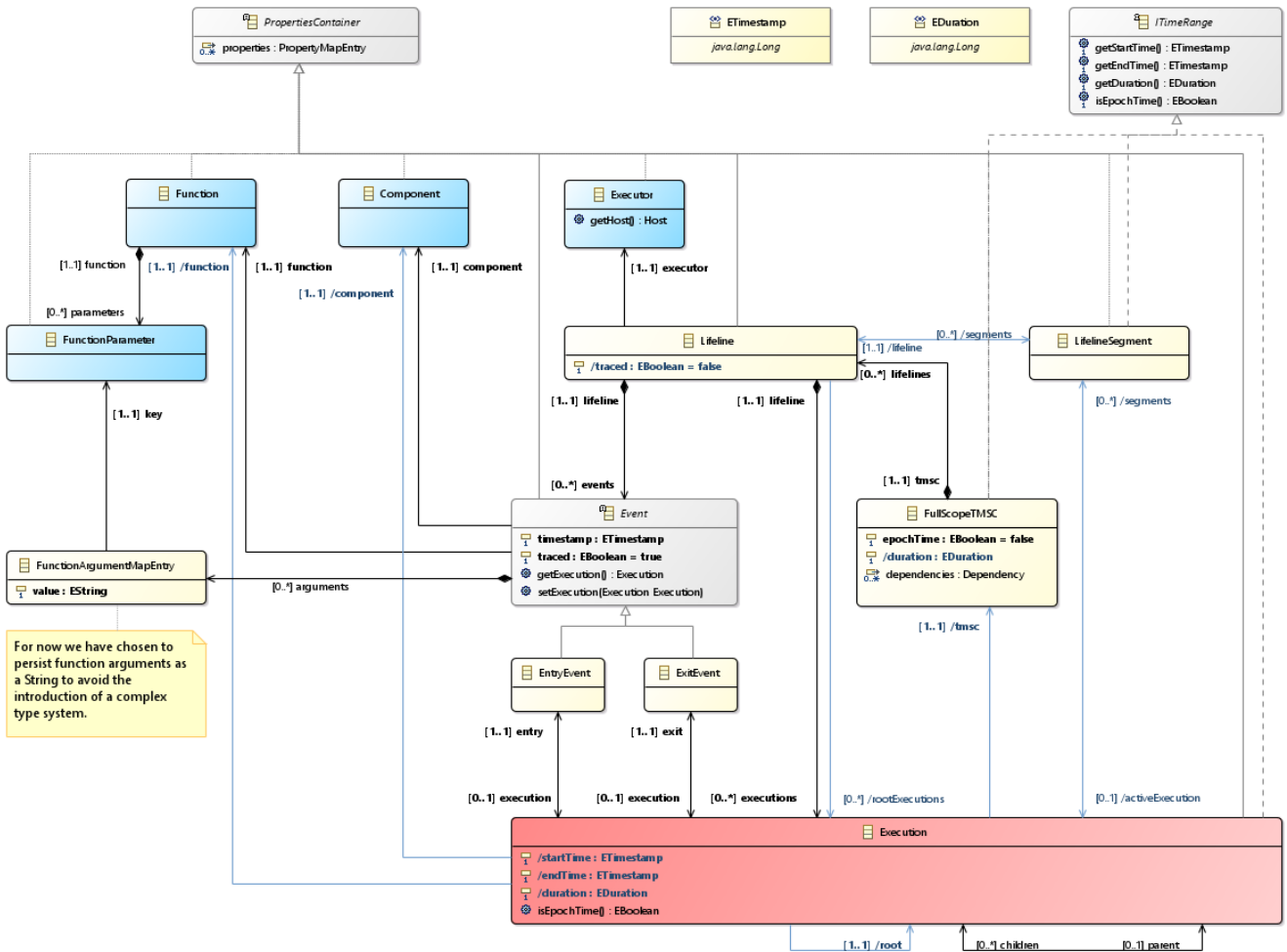


Figure 9. Diagram of call-stacks in the TMS meta-model

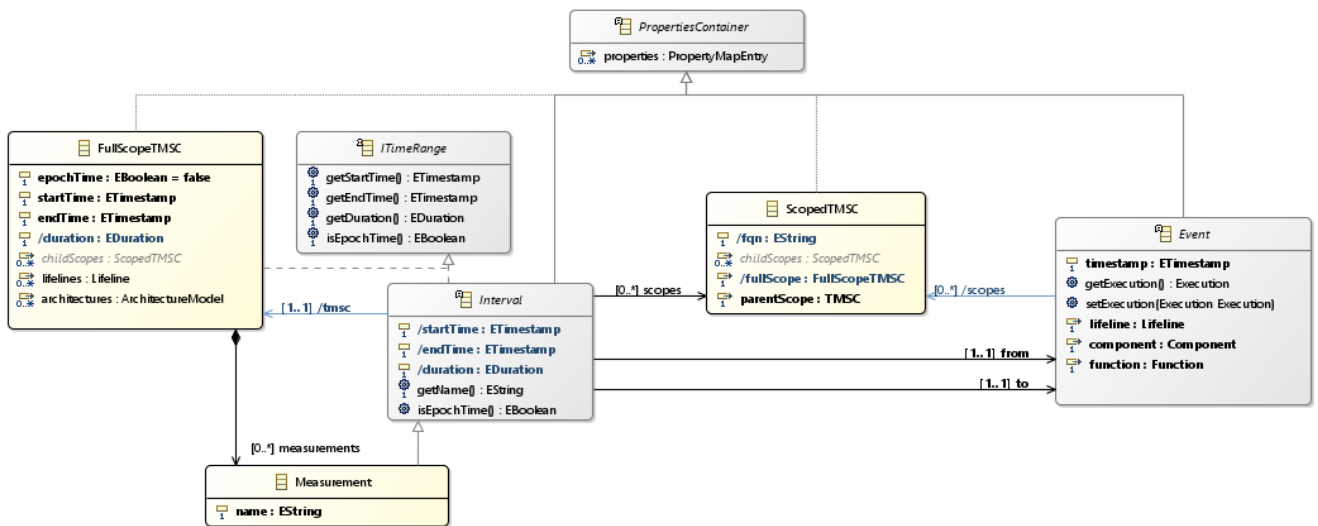


Figure 10. Diagram of measurements in the TMS meta-model

Ns Prefix

tmsc

Ns URI

<http://www.esi.nl/pps/tmsc>

4.8.1. Data Types

4.8.1.1. EDuration

Represents a duration in nanoseconds.

Instance Type Name

`java.lang.Long`

Used at

- `metric.Metric.budget`
- `metric.MetricInstance.duration`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Dependency.duration`
- `tmsc.Dependency.timeBound`
- `tmsc.DomainDependency.duration`
- `tmsc.DomainDependency.timeBound`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Execution.duration`
- `tmsc.FullScopeTMSD.duration`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Interval.duration`
- `tmsc.LifelineSegment.duration`
- `tmsc.LifelineSegment.timeBound`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Measurement.duration`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Message.duration`
- `tmsc.Message.timeBound`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.MessageControl.duration`
- `tmsc.MessageControl.timeBound`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Reply.duration`
- `tmsc.Reply.timeBound`
- `tmsc.ITimeRange.getDuration()`
- `tmsc.Request.duration`
- `tmsc.Request.timeBound`

- `tmesc.ITimeRange.getDuration()`

4.8.1.2. EID

Instance Type Name

`java.lang.String`

Used at

- `tmesc.ScopedTMSD.name`

4.8.1.3. ETimestamp

Represents a time stamp in nanoseconds. The timestamp can be either relative (i.e. relative to 0) or absolute (i.e. relative to the epoch). Please note that `{@link FullScopeTMSD#setEpochTime(boolean)}` should be set accordingly.

Instance Type Name

`java.lang.Long`

Used at

- `metric.MetricInstance.endTime`
- `metric.MetricInstance.startTime`
- `tmesc.ITimeRange.getEndTime()`
- `tmesc.ITimeRange.getStartTime()`
- `tmesc.Dependency.endTime`
- `tmesc.Dependency.startTime`
- `tmesc.DomainDependency.endTime`
- `tmesc.DomainDependency.startTime`
- `tmesc.ITimeRange.getEndTime()`
- `tmesc.ITimeRange.getStartTime()`
- `tmesc.EntryEvent.timestamp`
- `tmesc.Event.timestamp`
- `tmesc.Execution.endTime`
- `tmesc.Execution.startTime`
- `tmesc.ExitEvent.timestamp`
- `tmesc.FullScopeTMSD.endTime`
- `tmesc.FullScopeTMSD.startTime`
- `tmesc.ITimeRange.getEndTime()`
- `tmesc.ITimeRange.getStartTime()`
- `tmesc.Interval.endTime`
- `tmesc.Interval.startTime`

- `tmsc.LifelineSegment.endTime`
- `tmsc.LifelineSegment.startTime`
- `tmsc.ITimeRange.getEndTime()`
- `tmsc.ITimeRange.getStartTime()`
- `tmsc.Measurement.endTime`
- `tmsc.Measurement.startTime`
- `tmsc.ITimeRange.getEndTime()`
- `tmsc.ITimeRange.getStartTime()`
- `tmsc.Message.endTime`
- `tmsc.Message.startTime`
- `tmsc.ITimeRange.getEndTime()`
- `tmsc.ITimeRange.getStartTime()`
- `tmsc.MessageControl.endTime`
- `tmsc.MessageControl.startTime`
- `tmsc.ITimeRange.getEndTime()`
- `tmsc.ITimeRange.getStartTime()`
- `tmsc.Reply.endTime`
- `tmsc.Reply.startTime`
- `tmsc.ITimeRange.getEndTime()`
- `tmsc.ITimeRange.getStartTime()`
- `tmsc.Request.endTime`
- `tmsc.Request.startTime`
- `tmsc.ITimeRange.getEndTime()`
- `tmsc.ITimeRange.getStartTime()`

4.8.2. Types

4.8.2.1. Abstract Class Dependency

Super-types

- `properties.PropertiesContainer`
- `tmsc.ITimeRange`

Sub-types

- `tmsc.DomainDependency`
- `tmsc.LifelineSegment`
- `tmsc.Message`

- `tmsc.MessageControl`
- `tmsc.Reply`
- `tmsc.Request`

Attributes

Name	Type	Properties	Description
duration <code>tmsc.ITimeRange.getDuration()</code>	<code>tmsc.EDuration</code>	[1] unchangeable derived transient volatile	
endTime <code>tmsc.ITimeRange.getEndTime()</code>	<code>tmsc.ETimestamp</code>	[1] unchangeable derived transient volatile	
projection	EBoolean		If true, this dependency was not originally part of the TMSD, but is an effect of projection, see <code>nl.esi.pps.tmsc.util.TmscProjection</code> class. Projections should typically not be visible to the user, unless they provide additional information.
scheduled	EBooleanObject		If true, this dependency was induced by the (software) platform as result of scheduling. If false, this dependency reflects a control or data dependency as programmed in code. By default Executions and LifelineSegments-without-activeExecution are considered to be scheduled dependencies, see <code>TmscRefinements#refineWithCallsTacks(Lifeline)</code> .
startTime <code>tmsc.ITimeRange.getStartTime()</code>	<code>tmsc.ETimestamp</code>	[1] unchangeable derived transient volatile	

Name	Type	Properties	Description
<code>timeBound</code>	<code>tmsec.EDuration</code>		The time-bound specifies the lowerbound for the duration of this dependency. This analysis attribute is used in e.g. critical-path and slack analysis.

References

Name	Type	Properties	Description
<code>scopes</code>	<code>tmsec.ScopedTMSC</code> <i>EOpposite:</i> <code>dependencies</code>	non-resolveProxies unordered transient	All dependencies of a child-scope ScopedTMSC should be contained by its parent-scope TMSC. Note that this is a derived 'many' relation. Though the return type 'EList' provides methods to alter the content, no altering method should be used and will throw an UnsupportedOperationException upon usage.
<code>source</code>	<code>tmsec.Event</code> <i>EOpposite:</i> <code>fullScopeOutgoingDependencies</code>	non-resolveProxies <code>[1]</code> transient	
<code>target</code>	<code>tmsec.Event</code> <i>EOpposite:</i> <code>fullScopeIncomingDependencies</code>	non-resolveProxies <code>[1]</code> transient	
<code>tmsec</code>	<code>tmsec.FullScopeTMSC</code> <i>EOpposite:</i> <code>dependencies</code>	container <code>[1]</code>	

Operations

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>isEpochTime()</code> <code>tmsec.ITimeRange.isEpochTime()</code>	<i>returns</i> <code>EBoolean</code>		

Used at

- `tmsec.EntryEvent.fullScopeIncomingDependencies`

- `tmse.EntryEvent.fullScopeOutgoingDependencies`
- `tmse.Event.fullScopeIncomingDependencies`
- `tmse.Event.fullScopeOutgoingDependencies`
- `tmse.ExitEvent.fullScopeIncomingDependencies`
- `tmse.ExitEvent.fullScopeOutgoingDependencies`
- `tmse.FullScopeTMSD.dependencies`
- `tmse.ITMSD.getIncomingDependencies(event)`
- `tmse.ITMSD.getOutgoingDependencies(event)`
- `tmse.ITMSD.getDependencies()`
- `tmse.ITMSD.getIncomingDependencies(event)`
- `tmse.ITMSD.getOutgoingDependencies(event)`
- `tmse.ScopedTMSD.dependencies`
- `tmse.ITMSD.getIncomingDependencies(event)`
- `tmse.ITMSD.getOutgoingDependencies(event)`
- `tmse.ITMSD.getDependencies()`
- `tmse.ITMSD.getIncomingDependencies(event)`
- `tmse.ITMSD.getOutgoingDependencies(event)`

4.8.2.2. Class DomainDependency

Super-types

- `properties.PropertiesContainer`
- `tmse.Dependency`
- `tmse.ITimeRange`

4.8.2.3. Class EntryEvent

Super-types

- `properties.PropertiesContainer`
- `tmse.Event`

References

Name	Type	Properties	Description
execution ◻ <code>tmse.Event. getExecution()</code> ◻ <code>tmse.Event. setExecution(Execution)</code>	<code>tmse.Execution</code> <i>EOpposite:</i> <code>entry</code>	non- resolveProxies transient	

Used at

- `tmsec.Execution.entry`

4.8.2.4. Abstract Class Event

Super-types

- `properties.PropertiesContainer`

Sub-types

- `tmsec.EntryEvent`
- `tmsec.ExitEvent`

Attributes

Name	Type	Properties	Description
<code>timestamp</code>	<code>tmsec.ETimestamp</code>	[1]	
<code>traced</code>	<code>EBoolean</code>	[1] <i>Default:</i> <code>true</code>	

Containments

Name	Type	Properties	Description
<code>arguments</code>	<code>tmsec.FunctionArgument</code> <code>MapEntry</code>	<code>resolveProxies</code>	

References

Name	Type	Properties	Description
<code>component</code>	<code>specified.Component</code>	[1]	
<code>fullScopeIncomingDependencies</code>	<code>tmsec.Dependency</code> <i>EOpposite:</i> <code>target</code>	non- <code>resolveProxies</code>	
<code>fullScopeOutgoingDependencies</code>	<code>tmsec.Dependency</code> <i>EOpposite:</i> <code>source</code>	non- <code>resolveProxies</code>	
<code>function</code>	<code>implemented.Function</code>	[1]	
<code>lifeline</code>	<code>tmsec.Lifeline</code> <i>EOpposite:</i> <code>events</code>	container [1]	

Name	Type	Properties	Description
<code>scopes</code>	<code>tmsc.ScopedTMS</code>	non- resolveProxies unordered unchangeable derived transient volatile	Note that this is a derived 'many' relation. Though the return type 'EList' provides methods to alter the content, no altering method should be used and will throw an UnsupportedOperationException upon usage.
<code>tmsc</code>	<code>tmsc.FullScopeTMS</code>	non- resolveProxies [1] unchangeable derived transient volatile	

Operations

Name	Aspect and Type	Properties	Description
<code>abstract getExecution()</code> <code>[] tmsc.EntryEvent. execution</code> <code>[] tmsc.ExitEvent. execution</code>	returns <code>tmsc.Execution</code>		
<code>abstract setExecution(Execution)</code> <code>[] tmsc.EntryEvent. execution</code> <code>[] tmsc.ExitEvent. execution</code>	returns void <code>Execution tmsc.Execution</code>		

Used at

- `metric.MetricInstance.from`
- `metric.MetricInstance.to`
- `tmsc.Dependency.source`
- `tmsc.Dependency.target`
- `tmsc.DomainDependency.source`
- `tmsc.DomainDependency.target`
- `tmsc.ITMSC.getEvents()`
- `tmsc.ITMSC.getIncomingDependencies(event)`

- `tmSC.ITMSC.getOutgoingDependencies(event)`
- `tmSC.ITMSC.getEvents()`
- `tmSC.ITMSC.getIncomingDependencies(event)`
- `tmSC.ITMSC.getOutgoingDependencies(event)`
- `tmSC.Interval.from`
- `tmSC.Interval.to`
- `tmSC.Lifeline.events`
- `tmSC.LifelineSegment.source`
- `tmSC.LifelineSegment.target`
- `tmSC.Measurement.from`
- `tmSC.Measurement.to`
- `tmSC.Message.source`
- `tmSC.Message.target`
- `tmSC.MessageControl.source`
- `tmSC.MessageControl.target`
- `tmSC.Reply.source`
- `tmSC.Reply.target`
- `tmSC.Request.source`
- `tmSC.Request.target`
- `tmSC.ScopedTMSC.origin`
- `tmSC.ITMSC.getEvents()`
- `tmSC.ITMSC.getIncomingDependencies(event)`
- `tmSC.ITMSC.getOutgoingDependencies(event)`
- `tmSC.ITMSC.getEvents()`
- `tmSC.ITMSC.getIncomingDependencies(event)`
- `tmSC.ITMSC.getOutgoingDependencies(event)`

4.8.2.5. Class Execution

Execution should be considered final and should not be overridden. Executions are added by means of a refinement transformation, see [{@link nl.esi.pps.tmsc.transform.TmscRefinements#refineWithCallStacks\(FullScopeTMSC\)}](#)

Super-types

- `properties.PropertiesContainer`
- `tmSC.ITimeRange`

Attributes

Name	Type	Properties	Description
duration ◻ tmsc.ITimeRange. getDuration()	tmsc.EDuration	[1] unchangeable derived transient volatile	
endTime ◻ tmsc.ITimeRange. getEndTime()	tmsc.ETimestamp	[1] unchangeable derived transient volatile	
startTime ◻ tmsc.ITimeRange. getStartTime()	tmsc.ETimestamp	[1] unchangeable derived transient volatile	

References

Name	Type	Properties	Description
children	tmsc.Execution <i>EOpposite:</i> parent	non-resolveProxies	
component	specified.Component	non-resolveProxies [1] unchangeable derived transient volatile	
entry	tmsc.EntryEvent <i>EOpposite:</i> execution	non-resolveProxies [1]	
exit	tmsc.ExitEvent <i>EOpposite:</i> execution	non-resolveProxies [1]	

Name	Type	Properties	Description
function	implemented. Function	non- resolveProxies [1] unchangeable derived transient volatile	
lifeline	tmse.Lifeline <i>EOpposite:</i> executions	non- resolveProxies container [1] transient	
parent	tmse.Execution <i>EOpposite:</i> children	non- resolveProxies transient	Definition 2.3.5
root	tmse.Execution	non- resolveProxies [1] unchangeable derived transient volatile	
segments	tmse. LifelineSegment <i>EOpposite:</i> activeExecution	non- resolveProxies unchangeable derived transient volatile	Returns the LifeLineSegments for which this Execution is the active execution on its LifeLine. Note that this is a derived 'many' relation. Though the return type 'EList' provides methods to alter the content, no altering method should be used and will throw an UnsupportedOperationException upon usage.
tmse	tmse. FullScopeTMSC	non- resolveProxies [1] unchangeable derived transient volatile	

Operations

Name	Aspect and Type	Properties	Description
abstract isEpochTime() <code>↳ tmsc.ITimeRange.isEpochTime()</code>	<i>returns</i> EBoolean		

Used at

- `tmsc.EntryEvent.execution`
- `tmsc.Event.getExecution()`
- `tmsc.Event.setExecution(Execution)`
- `tmsc.Execution.children`
- `tmsc.Execution.parent`
- `tmsc.Execution.root`
- `tmsc.ExitEvent.execution`
- `tmsc.Lifeline.executions`
- `tmsc.Lifeline.rootExecutions`
- `tmsc.LifelineSegment.activeExecution`

4.8.2.6. Class ExitEvent

Super-types

- `properties.PropertiesContainer`
- `tmsc.Event`

References

Name	Type	Properties	Description
execution <code>↳ tmsc.Event.getExecution()</code> <code>↳ tmsc.Event.setExecution(Execution)</code>	<code>tmsc.Execution</code> <i>EOpposite: exit</i>	non-resolveProxies transient	

Used at

- `tmsc.Execution.exit`

4.8.2.7. Class FullScopeTMSC

FullScopeTMSC contains all events (via Lifeline) and dependencies of the TMSC model. It has the `startTime` and `endTime` attributes which are timestamps derived from the TMSC model, denoting the start and end time of the considered trace.

Super-types

- [properties.PropertiesContainer](#)
- [tmsc.ITMSC](#)
- [tmsc.ITimeRange](#)
- [tmsc.TMSC](#)

Attributes

Name	Type	Properties	Description
duration tmsc.ITimeRange.getDuration()	tmsc.EDuration	[1] unchangeable derived transient volatile	
endTime tmsc.ITimeRange.getEndTime()	tmsc.ETimestamp	[1]	
epochTime	EBoolean	[1] <i>Default: false</i>	
startTime tmsc.ITimeRange.getStartTime()	tmsc.ETimestamp	[1]	

Containments

Name	Type	Properties	Description
dependencies tmsc.ITMSC.getDependencies()	tmsc.Dependency <i>EOpposite: tmsc</i>		
lifelines	tmsc.Lifeline <i>EOpposite: tmsc</i>		
measurements	tmsc.Measurement	resolveProxies	

References

Name	Type	Properties	Description
architectures	architecture.ArchitectureModel		References all model roots (i.e. ArchitectureModels) that own the architectural elements that are referenced by the model elements of this FullScopeTMSC.

Used at

- [metric.Metric.tmscs](#)
- [metric.MetricInstance.tmsc](#)
- [tmsc.Dependency.tmsc](#)
- [tmsc.DomainDependency.tmsc](#)
- [tmsc.EntryEvent.tmsc](#)
- [tmsc.Event.tmsc](#)
- [tmsc.Execution.tmsc](#)
- [tmsc.ExitEvent.tmsc](#)
- [tmsc.FullScopeTMScope.fullScope](#)
- [tmsc.Interval.tmsc](#)
- [tmsc.Lifeline.tmsc](#)
- [tmsc.LifelineSegment.tmsc](#)
- [tmsc.Measurement.tmsc](#)
- [tmsc.Message.tmsc](#)
- [tmsc.MessageControl.tmsc](#)
- [tmsc.Reply.tmsc](#)
- [tmsc.Request.tmsc](#)
- [tmsc.ScopedTMScope.fullScope](#)
- [tmsc.TMScope.fullScope](#)

4.8.2.8. Class `FunctionArgumentMapEntry`

Instance Type Name

`java.util.Map$Entry`

Attributes

Name	Type	Properties	Description
<code>value</code>	<code>EString</code>	<code>[1]</code>	For now we have chosen to persist function arguments as a String to avoid the introduction of a complex type system.

References

Name	Type	Properties	Description
<code>key</code>	<code>implemented. FunctionParameter</code>	<code>[1]</code>	

Used at

- [tmsc.EntryEvent.arguments](#)

- `tmse.Event.arguments`
- `tmse.ExitEvent.arguments`

4.8.2.9. Interface ITMSC

Instance Type Name

`nl.esi.pps.tmse.ITMSC`

Sub-types

- `tmse.FullScopeTMSC`
- `tmse.ScopedTMSC`
- `tmse.TMSC`

Operations

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>getDependencies()</code> <code>[] tmse.FullScopeTMSC.dependencies</code> <code>[] tmse.ScopedTMSC.dependencies</code>	<i>returns</i> <code>tmse.Dependency</code>	unique	
<i>abstract</i> <code>getEvents()</code>	<i>returns</i> <code>tmse.Event</code>	unique	
<i>abstract</i> <code>getIncomingDependencies(event)</code>	<i>returns</i> <code>tmse.Dependency</code>	unique	
	<i>event</i> <code>tmse.Event</code>	[1]	
<i>abstract</i> <code>getOutgoingDependencies(event)</code>	<i>returns</i> <code>tmse.Dependency</code>	unique	
	<i>event</i> <code>tmse.Event</code>	[1]	

4.8.2.10. Interface ITimeRange

Instance Type Name

`nl.esi.pps.tmse.ITimeRange`

Sub-types

- `metric.MetricInstance`
- `tmse.Dependency`
- `tmse.DomainDependency`

- `tmsec.Execution`
- `tmsec.FullScopeTMSec`
- `tmsec.Interval`
- `tmsec.LifelineSegment`
- `tmsec.Measurement`
- `tmsec.Message`
- `tmsec.MessageControl`
- `tmsec.Reply`
- `tmsec.Request`

Operations

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>getDuration()</code> <code>tmsec.Dependency.duration</code> <code>tmsec.Execution.duration</code> <code>tmsec.FullScopeTMSec.duration</code> <code>tmsec.Interval.duration</code>	<i>returns</i> <code>tmsec.EDuration</code>	[1]	
<i>abstract</i> <code>getEndTime()</code> <code>tmsec.Dependency.endTime</code> <code>tmsec.Execution.endTime</code> <code>tmsec.FullScopeTMSec.endTime</code> <code>tmsec.Interval.endTime</code>	<i>returns</i> <code>tmsec.ETimestamp</code>	[1]	

Name	Aspect and Type	Properties	Description
<i>abstract</i> <code>getStartTime()</code> <code>▢ tmsc.Dependency. startTime</code> <code>▢ tmsc.Execution. startTime</code> <code>▢ tmsc.FullScopeTMS. startTime</code> <code>▢ tmsc.Interval.startTime</code>	<i>returns</i> <code>tmsc.ETimestamp</code>	[1]	
<i>abstract</i> <code>isEpochTime()</code> <code>▢ tmsc.Dependency. isEpochTime()</code> <code>▢ tmsc.Execution. isEpochTime()</code> <code>▢ tmsc.Interval. isEpochTime()</code>	<i>returns</i> <code>EBoolean</code>	[1]	

4.8.2.11. Abstract Class Interval

Super-types

- `properties.PropertiesContainer`
- `tmsc.ITimeRange`

Sub-types

- `metric.MetricInstance`
- `tmsc.Measurement`

Attributes

Name	Type	Properties	Description
<code>duration</code> <code>▢ tmsc.ITimeRange. getDuration()</code>	<code>tmsc.EDuration</code>	[1] unchangeable derived transient volatile	

Name	Type	Properties	Description
endTime □ <code>tmsc.ITimeRange. getEndTime()</code>	<code>tmsc.ETimestamp</code>	[1] unchangeable derived transient volatile	
startTime □ <code>tmsc.ITimeRange. getStartTime()</code>	<code>tmsc.ETimestamp</code>	[1] unchangeable derived transient volatile	

References

Name	Type	Properties	Description
from	<code>tmsc.Event</code>	[1]	
scopes	<code>tmsc.ScopedTMS</code>		Typically analysis stores its result as a ScopedTMS. This relation can be used to avoid duplicate analysis results in the model.
tmsc	<code>tmsc. FullScopeTMS</code>	[1] unchangeable derived transient volatile	
to	<code>tmsc.Event</code>	[1]	

Operations

Name	Aspect and Type	Properties	Description
abstract getName() □ <code>metric.MetricInstance. name</code> □ <code>tmsc.Measurement.name</code>	<i>returns</i> EString	[1]	
abstract isEpochTime() □ <code>tmsc.ITimeRange. isEpochTime()</code>	<i>returns</i> EBoolean		

4.8.2.12. Class Lifeline

Lifeline is the modeling element as defined in the UML Message Sequence Chart formalism and it is equivalent only to the instantiated nature (viewpoint) of the Component modeling element as defined in the abstract TMS meta-model. A lifeline contains the behavior of an Executor, in the

form of a sequence of events, over a specific time span.

Super-types

- [properties.PropertiesContainer](#)

Attributes

Name	Type	Properties	Description
traced	EBoolean	[1] unchangeable derived transient volatile	

Containments

Name	Type	Properties	Description
events	tmse.Event <i>EOpposite:</i> lifeline		
executions	tmse.Execution <i>EOpposite:</i> lifeline		

References

Name	Type	Properties	Description
executor	instantiated.Executor	[1]	
rootExecutions	tmse.Execution	non- resolveProxies unchangeable derived transient volatile	
segments	tmse.LifelineSegment <i>EOpposite:</i> lifeline	non- resolveProxies unchangeable derived transient volatile	Note that this is a derived 'many' relation. Though the return type 'EList' provides methods to alter the content, no altering method should be used and will throw an UnsupportedOperationException upon usage.

Name	Type	Properties	Description
tmsc	tmsc. FullScopeTMSCLifelines <i>EOpposite:</i> lifelines	container [1]	

Used at

- `tmsc.EntryEvent.lifeline`
- `tmsc.Event.lifeline`
- `tmsc.Execution.lifeline`
- `tmsc.ExitEvent.lifeline`
- `tmsc.FullScopeTMSCLifelines`
- `tmsc.LifelineSegment.lifeline`

4.8.2.13. Class LifelineSegment

Super-types

- `properties.PropertiesContainer`
- `tmsc.Dependency`
- `tmsc.ITimeRange`

References

Name	Type	Properties	Description
activeExecution	tmsc.Execution <i>EOpposite:</i> segments	non-resolveProxies unchangeable derived transient volatile	
lifeline	tmsc.Lifeline <i>EOpposite:</i> segments	non-resolveProxies [1] unchangeable derived transient volatile	

Used at

- `tmsc.Execution.segments`
- `tmsc.Lifeline.segments`

4.8.2.14. Class Measurement

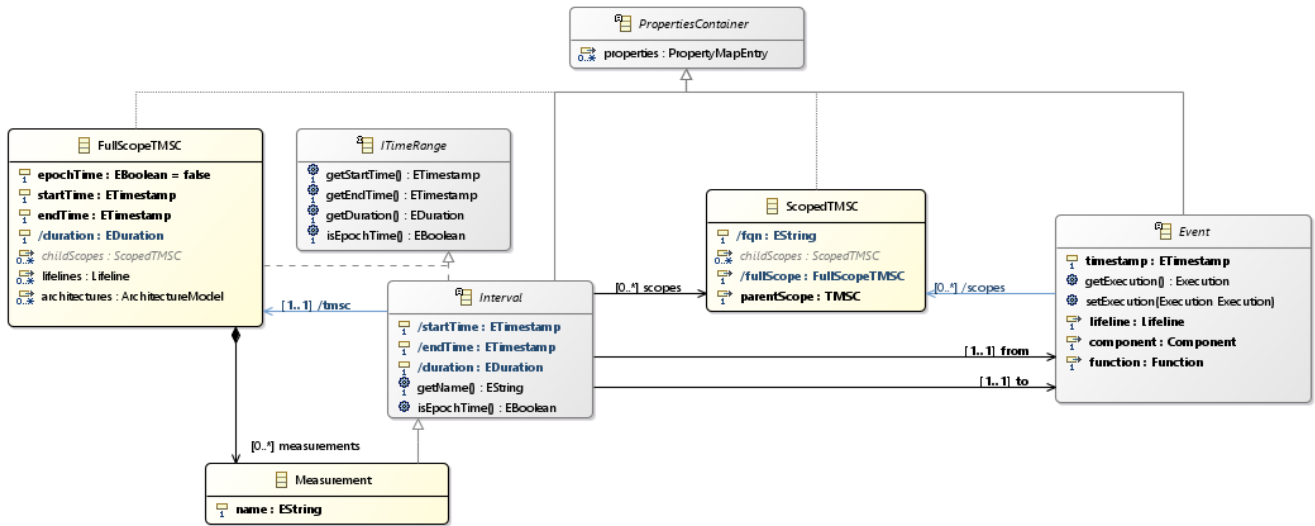


Figure 11. Diagram of measurements in the TMS meta-model

Super-types

- [properties.PropertiesContainer](#)
- [tmsc.ITimeRange](#)
- [tmsc.Interval](#)

Attributes

Name	Type	Properties	Description
name	EString	[1]	
tmsc.Interval.getName()			

Used at

- [tmsc.FullScopeTMS.measurements](#)

4.8.2.15. Class Message

Super-types

- [properties.PropertiesContainer](#)
- [tmsc.Dependency](#)
- [tmsc.ITimeRange](#)

Sub-types

- [tmsc.Reply](#)
- [tmsc.Request](#)

References

Name	Type	Properties	Description
<code>controlDependencies</code>	<code>tmsc.MessageControl</code> <i>EOpposite:</i> <code>message</code>	non- resolveProxies	

Used at

- `tmsc.MessageControl.message`

4.8.2.16. Class MessageControl

Super-types

- `properties.PropertiesContainer`
- `tmsc.Dependency`
- `tmsc.ITimeRange`

References

Name	Type	Properties	Description
<code>message</code>	<code>tmsc.Message</code> <i>EOpposite:</i> <code>controlDependencies</code>	non- resolveProxies <code>[1]</code> transient	

Used at

- `tmsc.Message.controlDependencies`
- `tmsc.Reply.controlDependencies`
- `tmsc.Request.controlDependencies`

4.8.2.17. Class Reply

Super-types

- `properties.PropertiesContainer`
- `tmsc.Dependency`
- `tmsc.ITimeRange`
- `tmsc.Message`

References

Name	Type	Properties	Description
<code>request</code>	<code>tmsc.Request</code> <i>EOpposite:</i> <code>replies</code>	non- resolveProxies transient	

Used at

- [tmsc.Request.replies](#)

4.8.2.18. Class Request

Super-types

- [properties.PropertiesContainer](#)
- [tmsc.Dependency](#)
- [tmsc.ITimeRange](#)
- [tmsc.Message](#)

References

Name	Type	Properties	Description
replies	tmsc.Reply <i>EOpposite:</i> request	non- resolveProxies	

Used at

- [tmsc.Reply.request](#)

4.8.2.19. Class ScopedTMSC

ScopedTMSC is a sub-graph of the whole TMSC model and defined for facilitating fragmentation of the TMSC model to sub-graphs (e.g. indication of a path), i.e. events and dependencies that hold specific properties, activities, etc. Each ScopedTMSC can be considered as a part of its parent scope TMSC and allows application of analysis techniques, i.e., critical path analysis, locally in a sub-graph. The dependencies of a child scope are contained by its parent scope.

Super-types

- [properties.PropertiesContainer](#)
- [tmsc.ITMSC](#)
- [tmsc.TMSC](#)

Attributes

Name	Type	Properties	Description
fqn	EString	[1] unchangeable derived transient volatile	
name	tmsc.EID	[1]	

References

Name	Type	Properties	Description
dependencies ◻ <code>tmsc.ITMSC. getDependencies()</code>	<code>tmsc.Dependency</code> <i>EOpposite:</i> <code>scopes</code>	non- resolveProxies	
origin	<code>tmsc.Event</code>		Typically a <code>ScopedTMSC</code> represents an analysis result that is created from a certain event or events, this is called the origin of the scope. Setting this attribute is optional, but may help successor analyses.
parentScope	<code>tmsc.TMSC</code> <i>EOpposite:</i> <code>childScopes</code>	container [1]	

Used at

- `metric.MetricInstance.scopes`
- `tmsc.Dependency.scopes`
- `tmsc.DomainDependency.scopes`
- `tmsc.EntryEvent.scopes`
- `tmsc.Event.scopes`
- `tmsc.ExitEvent.scopes`
- `tmsc.FullScopeTMSC.childScopes`
- `tmsc.Interval.scopes`
- `tmsc.LifelineSegment.scopes`
- `tmsc.Measurement.scopes`
- `tmsc.Message.scopes`
- `tmsc.MessageControl.scopes`
- `tmsc.Reply.scopes`
- `tmsc.Request.scopes`
- `tmsc.ScopedTMSC.childScopes`
- `tmsc.TMSC.childScopes`

4.8.2.20. Abstract Class TMSC

TMSC is a directed acyclic graph with events and dependencies, and some additional structural properties. A TMSC should at least contain all events that are referred to by its dependencies. Two specializations are considered the `FullScopeTMSC` and `ScopedTMSC`.

Super-types

- `properties.PropertiesContainer`
- `tmsc.ITMSC`

Sub-types

- `tmsc.FullScopeTMScope`
- `tmsc.ScopedTMScope`

Containments

Name	Type	Properties	Description
<code>childScopes</code>	<code>tmsc.ScopedTMScope</code> <i>EOpposite:</i> <code>parentScope</code>	<i>EKeys:</i> <code>name</code>	All dependencies of a child scope should also be contained by its parent scope.

References

Name	Type	Properties	Description
<code>fullScope</code>	<code>tmsc.FullScopeTMScope</code>	<code>non-resolveProxies</code> <code>[1]</code> <code>unchangeable</code> <code>derived</code> <code>transient</code> <code>volatile</code>	

Used at

- `tmsc.ScopedTMScope.parentScope`

[1] H. Koziol, "Performance Evaluation of Component-based Software Systems: A Survey", Elsevier, 2010.